Established – 1961 Subject:_____

# SEVA SADAN'S

# R. K. TALREJA COLLEGE

# OF

# ARTS, SCIENCE & COMMERCE ULHASNAGAR – 421003

# CERTIFICATE

**This is to certify that Mr./Ms. _____ of S.Y. Information Technology (SYIT) Roll No. _____ has satisfactorily completed the Open Source DataBase Management System Mini Project entitled _____ during the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.**

**PROF. INCHARGE** **HEAD OF DEPT**

_____ _____

# INDEX


| Sr. No. | Chapter Title | PAGE NO |
|---|---|---|
| 1 | Introduction | |
| 2 | Problem Definition | |
| 3 | Objectives of the Project | |
| 4 | Scope of the Project | |
| 5 | Requirement Specification | |
| 6 | System Design | |
| 7 | Database Design | |
| 8 | UML Diagrams | |
| 9 | SQL Implementation | |
| 10 | System Testing and Result | |
| 11 | Security, Backup and Recovery | |
| 12 | Future Scope and Conclusion | |
| 13 | References | |
| 14 | Glossary | |

# CHAPTER 1: INTRODUCTION

## 1.1 What the Database System Is

The Sales Order Processing System is a relational database system designed to manage customer information, product details, and sales transactions in a structured manner. The system is implemented using MySQL and consists of tables such as customer, product, sales_order, and order_item. These tables are connected using primary and foreign keys to maintain relationships and ensure data integrity.

The system allows storage, retrieval, and updating of sales-related data using SQL queries. It also supports transaction management using START TRANSACTION, COMMIT, and ROLLBACK, ensuring reliable and consistent order processing. Overall, the database system provides an organized and efficient way to handle business sales operations.

## 1.2 What Problem Area It Addresses

Many small businesses manage sales records manually using paper registers or spreadsheets. This approach often leads to data duplication, stock mismanagement, calculation errors, and difficulty in tracking orders. There is also a high risk of inconsistency when multiple users update records simultaneously.

The Sales Order Processing System addresses these problems by providing a centralized database solution. It ensures accurate stock management, structured data storage, and proper relationship between customers and orders. By using constraints and transaction control, the system improves reliability and reduces human errors.

## 1.3 Why MySQL Is Used as an Open-Source Solution

MySQL is used in this project because it is a powerful and widely used open-source relational database management system. It supports SQL for data manipulation and provides features such as constraints, indexing, and transaction management. These features ensure data accuracy and consistency.

Since MySQL is free and compatible with multiple operating systems, it is suitable for academic projects and small business applications. Its support for transaction control and concurrency management makes it ideal for implementing the Sales Order Processing System efficiently.

# CHAPTER 2: PROBLEM DEFINITION

## 2.1 Existing Manual / Unstructured System

In many businesses, sales orders are managed manually using paper records or spreadsheets. Customer details, product information, and stock records are stored separately without proper integration. This unstructured system makes order processing slow and increases the chances of human errors. Tracking previous orders, verifying stock availability, and updating records require manual effort, which consumes time and reduces efficiency. As business transactions increase, maintaining accuracy becomes even more difficult. There is no centralized system to monitor all activities in real time.

## 2.2 Issues in the Existing System

The manual system faces several problems that affect business operations:

- Data redundancy due to repeated customer and product entries
- Data inconsistency between stock records and sales transactions
- Lack of data integrity and relationship control
- No proper security or restricted access to sensitive data
- No transaction mechanism to undo errors or failed operations
- Difficulty in generating reports and analyzing sales performance

These issues can lead to financial losses, incorrect stock levels, and poor decision-making. Without automation, the organization cannot ensure reliability and accuracy in its sales operations.

## 2.3 Need for a Database-Driven Solution

A database-driven system is required to organize and manage sales data efficiently and accurately. The Sales Order Processing System uses MySQL to store customer, product, and order details in structured relational tables. It ensures data integrity through constraints such as PRIMARY KEY and FOREIGN KEY and maintains consistency using transaction control mechanisms like COMMIT and ROLLBACK. The system also supports controlled stock updates using row-level locking to prevent concurrency issues. This improves accuracy, security, scalability, and overall performance of sales operations, making it suitable for modern business environments.

# CHAPTER 3: OBJECTIVES OF THE PROJECT

### 3.1 To Design a Structured Database System

The main objective of this project is to design a structured and organized database system for managing sales operations. The system uses relational tables such as customer, product, sales_order, and order_item to store data efficiently. These tables are connected using proper relationships to ensure smooth data flow and avoid redundancy. The structured design improves data organization and makes the system reliable for handling business transactions.

### 3.2 To Implement SQL Queries for Data Management

Another objective is to implement SQL queries for performing various database operations. The project uses SQL commands like CREATE, INSERT, SELECT, UPDATE, and JOIN to manage and retrieve data. Transaction control commands such as START TRANSACTION, COMMIT, and ROLLBACK are also implemented to ensure accurate and safe order processing. This helps in understanding real-world database operations**.**

### 3.3 To Ensure Data Integrity Using Constraints

The project aims to maintain data accuracy and consistency by using constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE. These constraints prevent duplicate records and ensure that every entry in the database is valid. Primary keys uniquely identify each record, while foreign keys maintain proper relationships between related tables such as customers and sales orders.

The NOT NULL constraint ensures that important fields like product name and price cannot be left empty. The UNIQUE constraint avoids duplication of sensitive fields such as customer email addresses. In addition, transaction control mechanisms like COMMIT and ROLLBACK protect the database from incomplete or incorrect updates. Together, these features maintain reliability, consistency, and integrity of data within the system.

### 3.4 To Gain Hands-On Experience with MySQL

This project provides practical exposure to MySQL and relational database concepts. It helps in understanding database creation, table design, data insertion, and query execution. Students learn how to manage transactions and handle concurrency issues using features like row-level locking.

By implementing real-world operations such as order placement and stock updates, learners gain a clear understanding of how database systems work in business environments. The project also improves problem-solving skills related to data management. Overall, it strengthens technical knowledge and prepares students for future careers in database administration and software development.

# CHAPTER 4: SCOPE OF THE PROJECT

## 4.1 Where the System Can Be Used

The Sales Order Processing System can be used in various business environments where products are sold and customer orders need to be managed efficiently. It is suitable for retail shops, wholesalers, small businesses, and startups that require a structured method to store customer details, product information, and sales transactions. The system helps in maintaining accurate stock levels and tracking order history in an organized manner.

The database system can also be implemented in online businesses where orders are placed digitally and inventory must be updated automatically. Since the system is built using MySQL, it can be easily integrated with web or desktop applications for real-time order processing. The structured relational design ensures scalability, allowing businesses to expand their operations without losing data consistency. Thus, the system has practical application in both small-scale and growing business environments.

## 4.2 Who Are the Users

The system can be used by different types of users depending on the organizational structure. The primary user is the **Administrator**, who has full control over database operations such as adding products, managing customers, processing orders, and maintaining stock records. The administrator ensures that data is accurate and up to date.

Business staff or sales employees can also use the system to enter customer orders and check product availability. In an academic environment, students and instructors can use the system for learning database concepts and practicing SQL queries. The system may also support business managers who require sales reports and stock summaries for decision-making. Therefore, the system is designed to be useful for administrators, staff members, students, and business users.

In educational institutions, students can be users of the system to view assigned equipment such as lab devices or library assets.

## 4.3 Academic Limitations

The Sales Order Processing System is primarily developed for academic and learning purposes. It focuses on demonstrating core database concepts such as table creation, relationships, constraints, transaction management, and concurrency control. The system does not include a graphical user interface (GUI) or web-based frontend, as the main objective is to understand backend database operations using MySQL.

The project operates on a limited dataset for demonstration and testing purposes. It does not handle large-scale enterprise-level transactions or high user traffic scenarios. Advanced features such as payment processing, invoice generation, automated reporting dashboards, and integration with external systems are not included in this version.

Security implementation is basic and limited to database-level constraints and transaction control. Real-world systems would require stronger authentication, encryption, and role-based access mechanisms. Despite these limitations, the project effectively fulfills its academic goal of providing practical knowledge of relational

# CHAPTER 5: REQUIREMENT SPECIFICATION

**Hardware Requirements:**

| **Component** | **Specification** |
|---|---|
| Computer / Laptop | Standard desktop or laptop system |
| Processor | Intel i3 or higher |
| RAM | Minimum 4 GB (8 GB recommended) |
| Storage | Minimum 20 GB free disk space |
| Keyboard & Mouse | Required for system operation |

**Software Requirements:**

| **Software** | **Purpose** |
|---|---|
| MySQL Server | Database management |
| MySQL Workbench | Query execution |
| OS (Windows/Linux) | Platform |
| SQL | Query language |

# CHAPTER 6: SYSTEM DESIGN

## 6.1 System Architecture (Conceptual Design)

The Sales Order Processing System follows a simple client–server architecture based on a relational database model. In this system, the user interacts with the database through SQL queries using tools such as MySQL Workbench. The MySQL Server acts as the backend component that processes these queries, manages stored data, and ensures consistency and integrity of transactions.

When a user executes a query such as inserting a new sales order or retrieving customer details, the request is sent to the MySQL Server. The server processes the query, checks constraints (such as PRIMARY KEY and FOREIGN KEY), executes the required operations, and returns the result to the user. This structured interaction ensures controlled access and proper management of data.

The system architecture mainly consists of three conceptual layers:

• **User Layer** – The user (administrator or staff) interacts with the system by writing and executing SQL queries.

• **Application/Query Layer** – SQL statements such as CREATE, INSERT, SELECT, UPDATE, and transaction commands are used to communicate with the database.

• **Database Layer (MySQL Server)** – Stores structured tables (customer, product, sales_order, order_item) and manages relationships, constraints, and transaction control.

**Query Execution Flow**

The user enters an SQL command (e.g., placing an order).

The MySQL Server receives and validates the query.

Constraints and relationships are checked to maintain data integrity.

If the operation is transactional, the system uses START TRANSACTION.

If all operations succeed, COMMIT is executed; otherwise, ROLLBACK restores the previous state.

The result is displayed to the user.

This design ensures that all database operations are controlled, consistent, and secure. The use of transaction management and row-level locking (SELECT ... FOR UPDATE) provides concurrency control, preventing multiple users from updating the same record simultaneously.

Overall, the system design is simple, reliable, and focused on backend database management. It demonstrates how structured query execution and transaction handling maintain data consistency in a Sales Order Processing environment.

---

# CHAPTER 7: DATABASE DESIGN

**CHAPTER 7: DATABASE DESIGN**

---

### 7.1 Entity Description

The Sales Order Processing System is designed using a relational database model consisting of four main entities: **Customer, Product, Sales_Order, and Order_Item**. Each entity represents a specific component of the sales process and is stored as a separate table in the database.

The **Customer** entity stores information related to customers who place orders. It includes details such as customer name, email, and phone number. Each customer is uniquely identified by a customer_id.

The **Product** entity maintains details about products available for sale. It includes product name, price, and stock quantity. The stock attribute helps in managing inventory and ensuring that orders are processed only when products are available.

The **Sales_Order** entity records transaction details whenever a customer places an order. It contains information such as order date and total amount. This table connects customers to their respective orders using a foreign key.

The **Order_Item** entity stores details of products included in each order. It links the Sales_Order and Product tables using foreign keys. This structure allows one order to contain multiple products, thereby maintaining a one-to-many relationship between sales orders and order items.

---

### 7.2 Table Structure

Below are the table structures used in the system:

**Customer Table**

| Attribute | Data Type | Description |
|---|---|---|
| customer_id | INT (PK, AUTO_INCREMENT) | Unique ID of customer |
| customer_name | VARCHAR(50) | Name of customer |
| customer_email | VARCHAR(50) | Email address (Unique) |
| customer_phone | VARCHAR(15) | Contact number |

---

**Product Table**

| Attribute | Data Type | Description |
|---|---|---|
| product_id | INT (PK, AUTO_INCREMENT) | Unique ID of product |
| product_name | VARCHAR(50) | Name of product |
| price | DECIMAL(10,2) | Product price |
| stock | INT | Available quantity |

---

**Sales_Order Table**

| Attribute | Data Type | Description |
|---|---|---|
| order_id | INT (PK, AUTO_INCREMENT) | Unique ID of order |
| customer_id | INT (FK) | References customer table |
| order_date | DATE | Date of order |
| total_amount | DECIMAL(10,2) | Total order value |

---

**Order_Item Table**

| Attribute | Data Type | Description |
|---|---|---|
| order_item_id | INT (PK, AUTO_INCREMENT) | Unique ID of order item |
| order_id | INT (FK) | References sales_order table |
| product_id | INT (FK) | References product table |
| quantity | INT | Quantity ordered |
| price | DECIMAL(10,2) | Price per product |

---

### 7.3 Constraints Used

The database uses several constraints to maintain data integrity and consistency.

• **PRIMARY KEY** – Ensures that each record in a table is uniquely identified. For example, customer_id, product_id, order_id, and order_item_id uniquely identify records in their respective tables.

• **FOREIGN KEY** – Establishes relationships between tables. For example, customer_id in the sales_order table references the customer table, ensuring that every order belongs to a valid customer.

• **NOT NULL** – Prevents important fields such as product_name and price from being left empty. This ensures mandatory data entry.

• **UNIQUE** – Prevents duplicate entries in specific columns such as customer_email, ensuring that each customer has a unique email address.

These constraints help maintain accuracy, prevent invalid data entry, and enforce proper relationships between tables. They ensure that the Sales Order Processing System operates reliably and efficiently.

---

# CHAPTER 8: UML DIAGRAMS

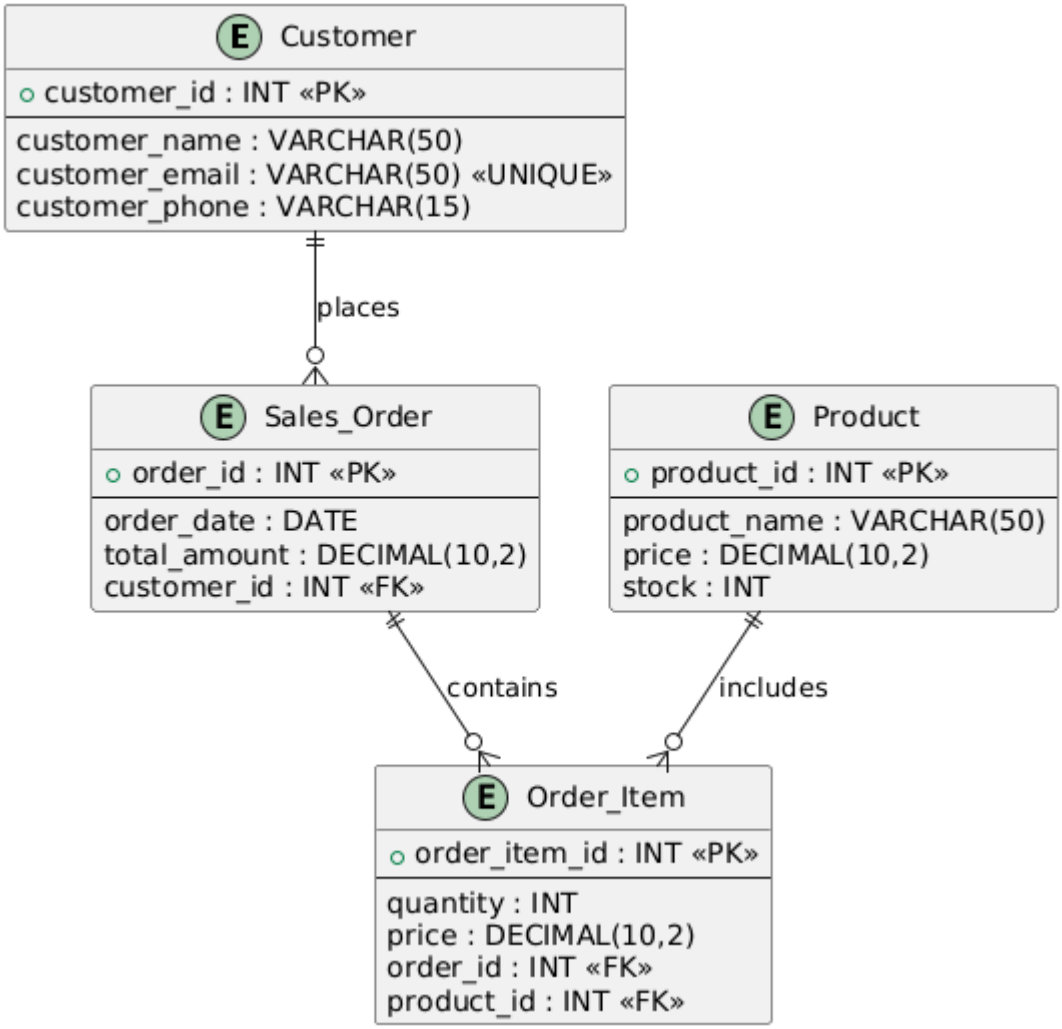



**Figure 8.1: ER[Entity Relationship] Diagram**

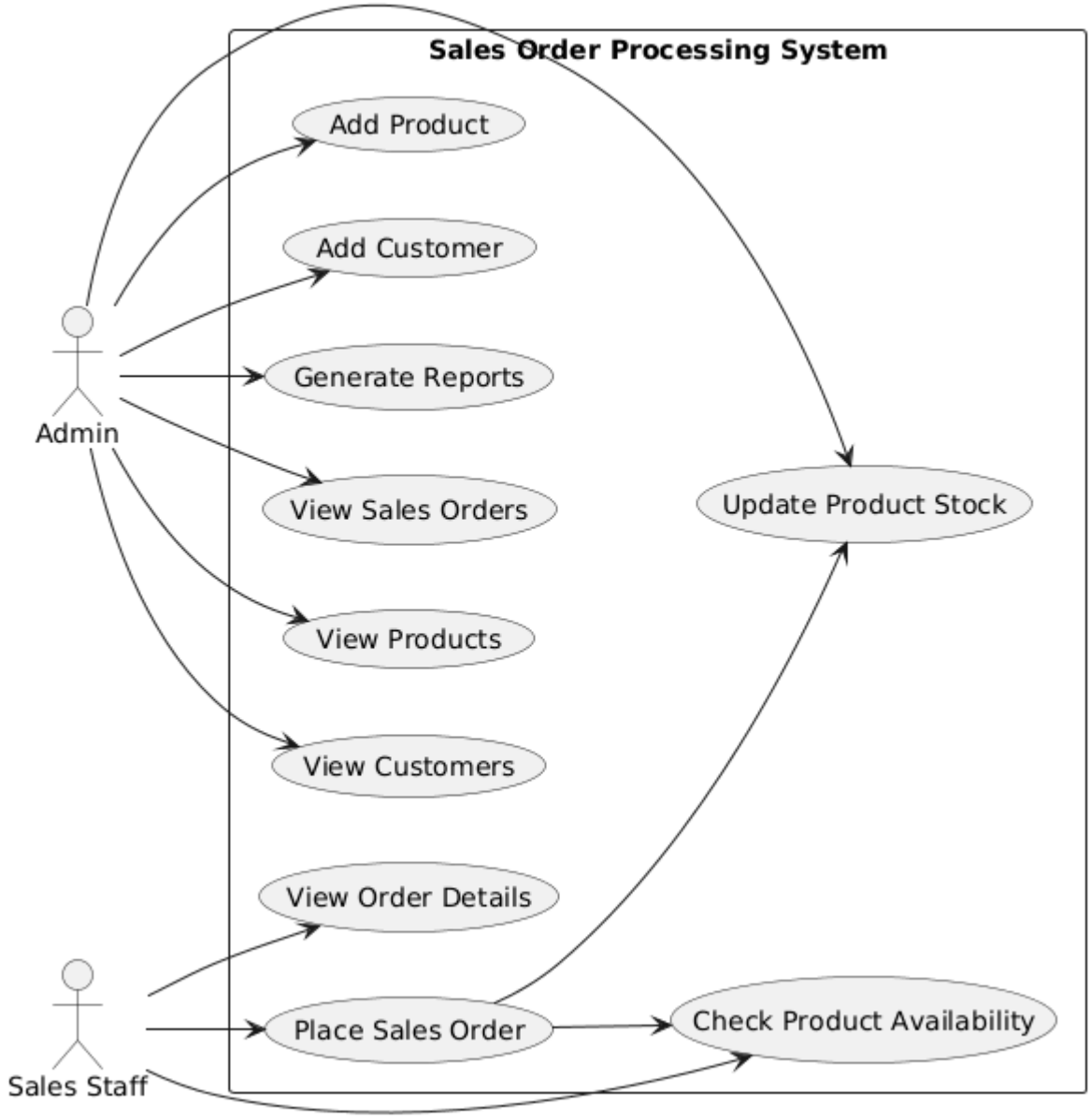



**Figure 8.2: USE CASE Diagram**

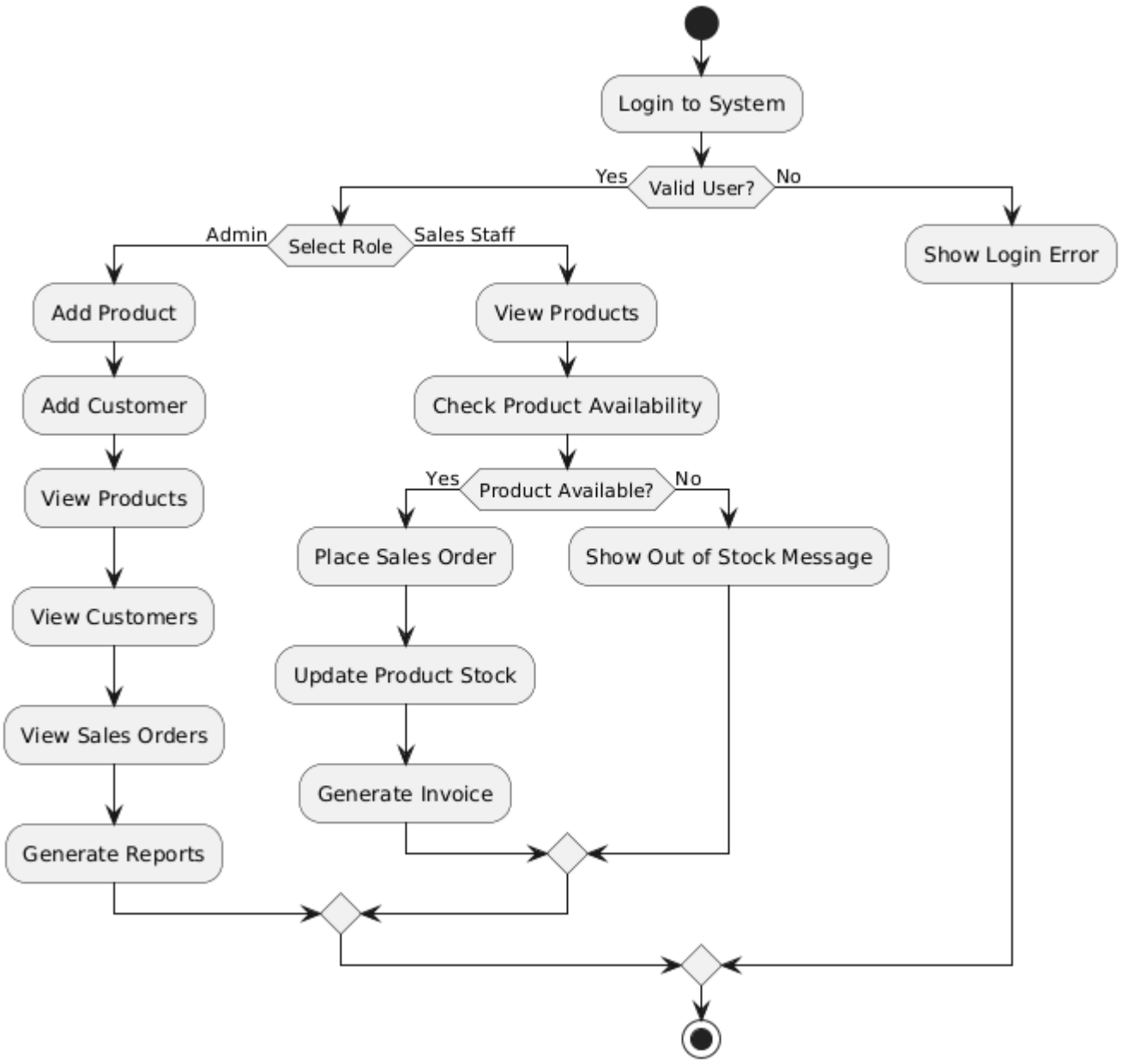



**Figure 8.3: Activity Diagram**

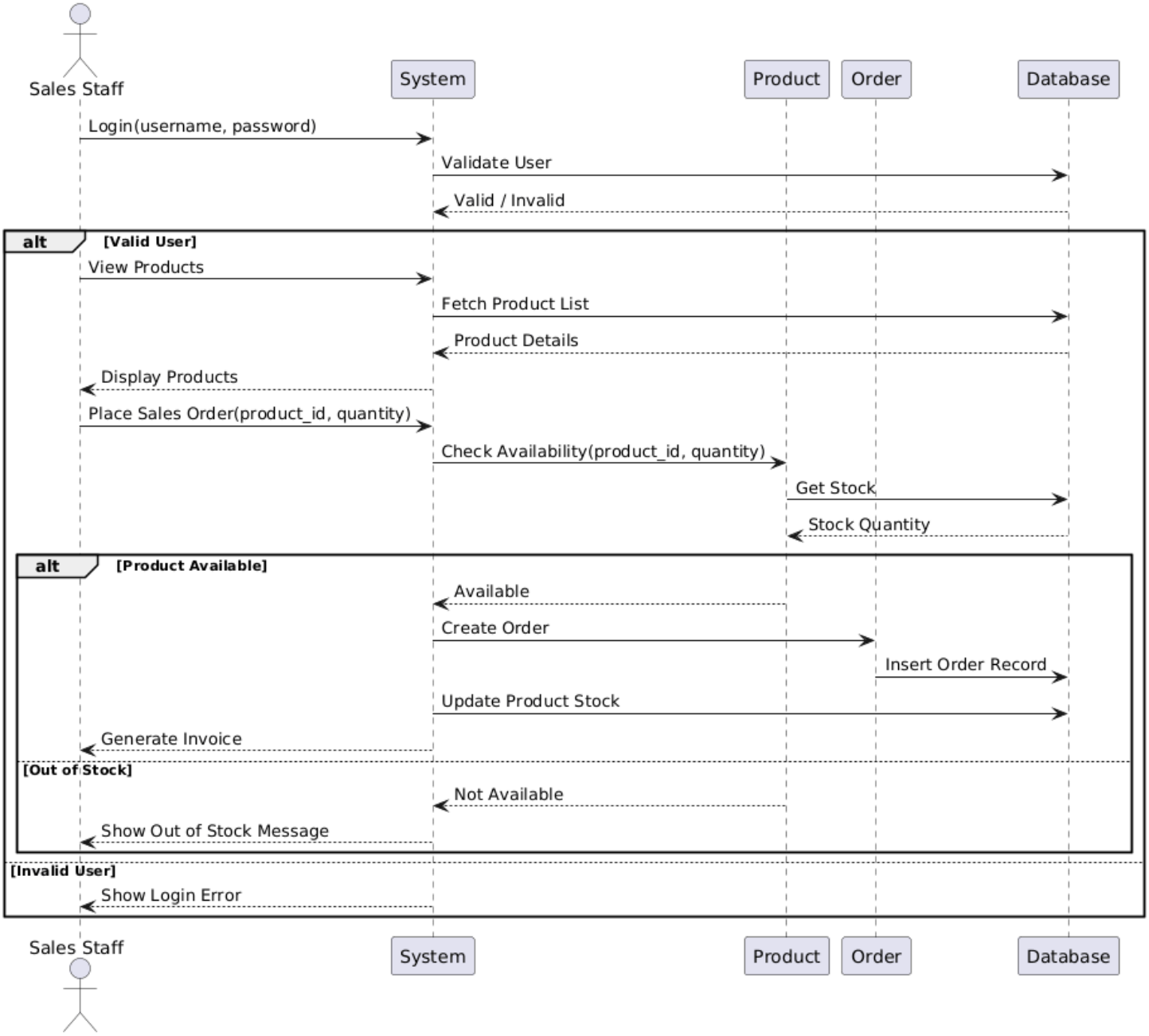



**Figure 8.4: Sequence Diagram**

# CHAPTER 9: SQL IMPLEMENTATION

---

## 9.1 Database Creation

```
CREATE DATABASE sales_order_processing;
USE sales_order_processing;
```

---

## 9.2 Table Creation

### Customer Table

```
CREATE TABLE customer (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL,
    customer_email VARCHAR(50) UNIQUE,
    customer_phone VARCHAR(15)
);
```

---

### Product Table

```
CREATE TABLE product (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL
);
```

---

### Sales Order Table

```
CREATE TABLE sales_order (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);
```

---

### Order Item Table

```
CREATE TABLE order_item (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES sales_order(order_id),
    FOREIGN KEY (product_id) REFERENCES product(product_id)
);
```

---

## 9.3 Data Insertion

### Insert 5 Customers

```
INSERT INTO customer (customer_name, customer_email, customer_phone)
VALUES
('Amit Shah', 'amit@gmail.com', '9876543210'),
('Neha Verma', 'neha@gmail.com', '9123456789'),
```

```
('Rahul Mehta', 'rahul@gmail.com', '9988776655'),
('Pooja Singh', 'pooja@gmail.com', '9090909090'),
('Karan Patel', 'karan@gmail.com', '9012345678');
```

---

**Insert 5 Products**

```
INSERT INTO product (product_name, price, stock)
VALUES
('Laptop', 55000, 10),
('Printer', 12000, 5),
('Mouse', 500, 50),
('Keyboard', 800, 30),
('Monitor', 15000, 8);
```

---

**Insert 5 Sales Orders**

```
INSERT INTO sales_order (customer_id, order_date, total_amount)
VALUES
(1, CURDATE(), 55000),
(2, CURDATE(), 12000),
(3, CURDATE(), 1500),
(4, CURDATE(), 16000),
(5, CURDATE(), 70000);
```

---

**Insert 5 Order Items**

```
INSERT INTO order_item (order_id, product_id, quantity, price)
VALUES
(1, 1, 1, 55000),
(2, 2, 1, 12000),
(3, 3, 3, 1500),
(4, 5, 1, 15000),
(5, 1, 1, 55000);
```

---

**9.4 Data Retrieval Queries**

**SELECT with WHERE**

```
SELECT *
FROM product
WHERE stock > 0;
```

---

**SELECT with JOIN**

```
SELECT o.order_id, c.customer_name, o.order_date, o.total_amount
FROM sales_order o
JOIN customer c ON o.customer_id = c.customer_id;
```

---

**JOIN Multiple Tables**

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price
FROM order_item oi
JOIN product p ON oi.product_id = p.product_id
JOIN sales_order o ON oi.order_id = o.order_id;
```

---

**9.5 Advanced Queries**

**GROUP BY**

```
SELECT product_id, SUM(quantity) AS total_quantity_sold
FROM order_item
GROUP BY product_id;
```

---

**HAVING**

```
SELECT product_id, SUM(quantity) AS total_quantity_sold
FROM order_item
GROUP BY product_id
HAVING SUM(quantity) > 1;
```

---

**Subquery**

```
SELECT customer_name
FROM customer
WHERE customer_id IN (
   SELECT customer_id
   FROM sales_order
);
```

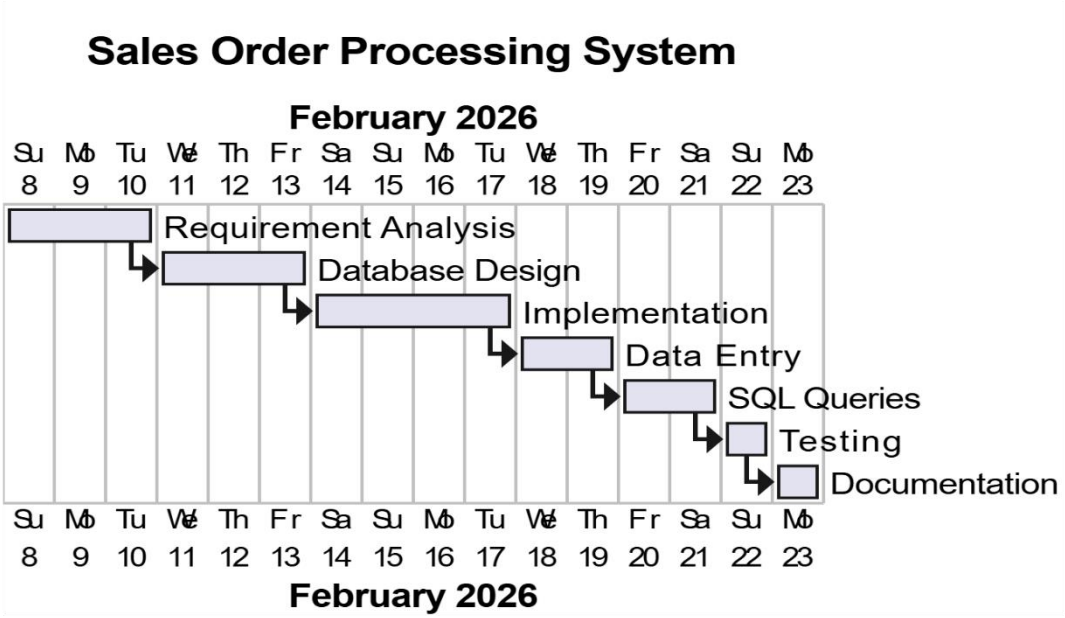
---

**View Creation**

```
CREATE VIEW sales_report_view AS
SELECT c.customer_name, p.product_name, oi.quantity, oi.price
FROM order_item oi
JOIN sales_order o ON oi.order_id = o.order_id
JOIN customer c ON o.customer_id = c.customer_id
JOIN product p ON oi.product_id = p.product_id;
```

**SELECT * FROM sales_report_view;**

# GANTT CHATS



# CHAPTER 10: SYSTEM TESTING AND RESULT

The Sales Order Processing System was successfully designed using SQL and database management concepts. It efficiently manages customer details, product information, sales orders, and order items in a structured manner. The system ensures data accuracy and integrity through the use of primary and foreign key constraints. It supports basic transaction handling and report generation using SQL queries. Overall, the system meets the required objectives and provides a reliable solution for managing sales operations in a small-scale business environment.

## 10.1 Query Testing

All essential SQL queries were tested to verify the correctness of database operations. The following types of queries were executed:

- DDL Queries (CREATE, DROP) to test database and table creation
- DML Queries (INSERT, UPDATE, DELETE) to test data manipulation
- DQL Queries (SELECT) to retrieve and analyze data

Each query was tested using:

- Valid inputs to ensure correct execution
- Invalid inputs to check error handling and constraint enforcement

All queries executed successfully and returned the expected results. This confirmed that the database schema, relationships, and query logic were implemented correctly and functioned as designed.

1. **Database and Table Creation Testing**

   Database and Table Creation Testing is performed to verify that the database and all required tables are created successfully without errors. This testing ensures that the database structure is correctly defined before

inserting or retrieving any data.

**Actual Result**

The database sales_order_processing (customer, product, sales_order, order_item) were created successfully.

**Customer Table**

```
mysql> select*from customer;
+-------------+---------------+-----------------+----------------+
| customer_id | customer_name | customer_email  | customer_phone |
+-------------+---------------+-----------------+----------------+
|           1 | Amit Shah     | amit@gmail.com  | 9876543210     |
|           2 | Neha Verma    | neha@gmail.com  | 9123456789     |
|           8 | Rahul Mehta   | rahul@gmail.com | 9988776655     |
|           9 | Pooja Singh   | pooja@gmail.com | 9090909090     |
|          10 | Karan Patel   | karan@gmail.com | 9012345678     |
+-------------+---------------+-----------------+----------------+
```

**Product Table**

```
mysql> select*from product;
+------------+--------------+----------+-------+
| product_id | product_name | price    | stock |
+------------+--------------+----------+-------+
|          1 | Laptop       | 55000.00 |     9 |
|          2 | Printer      | 12000.00 |     5 |
|          3 | Mouse        |   500.00 |    50 |
|          4 | Laptop       | 55000.00 |    10 |
|          5 | Printer      | 12000.00 |     5 |
|          6 | Mouse        |   500.00 |    50 |
|          7 | Keyboard     |   800.00 |    30 |
|          8 | Monitor      | 15000.00 |     8 |
+------------+--------------+----------+-------+
```

**Sales_order Table**

```
mysql> select*from sales_order;
+----------+-------------+------------+--------------+
| order_id | customer_id | order_date | total_amount |
+----------+-------------+------------+--------------+
|        1 |           1 | 2026-02-21 |     55000.00 |
|        2 |           2 | 2026-02-21 |     12000.00 |
|        3 |           3 | 2026-02-21 |      1500.00 |
|        4 |           4 | 2026-02-21 |     16000.00 |
|        5 |           5 | 2026-02-21 |     70000.00 |
|        6 |           1 | 2026-02-21 |     55000.00 |
|        7 |           2 | 2026-02-21 |     12000.00 |
|        8 |           3 | 2026-02-21 |      1500.00 |
|        9 |           4 | 2026-02-21 |     16000.00 |
|       10 |           5 | 2026-02-21 |     70000.00 |
+----------+-------------+------------+--------------+
```

**order_item Table**

```
mysql> select*from order_item;
+---------------+----------+------------+----------+----------+
| order_item_id | order_id | product_id | quantity | price    |
+---------------+----------+------------+----------+----------+
|             1 |        1 |          1 |        1 | 55000.00 |
|             2 |        2 |          2 |        1 | 12000.00 |
|             3 |        3 |          3 |        3 |  1500.00 |
|             4 |        4 |          5 |        1 | 15000.00 |
|             5 |        5 |          1 |        1 | 55000.00 |
+---------------+----------+------------+----------+----------+
5 rows in set (0.00 sec)
```

s

**2. Join Query**

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price
FROM order_item oi
JOIN product p ON oi.product_id = p.product_id
JOIN sales_order o ON oi.order_id = o.order_id;
```

**Result Outcome**

- The query executed successfully without any errors.
- Data was retrieved from the **Sales Order**, **Order Item**, and **Product** tables.
- The output displayed the following details:
  - Order ID
  - Product Name
  - Quantity
  - Price
- JOIN operations correctly linked the tables using **Order ID** and **Product ID**.
- The retrieved results matched the data stored in the database tables.
- This confirms that the database relationships and JOIN query logic are functioning correctly.

```
+----------+--------------+----------+----------+
| order_id | product_name | quantity | price    |
+----------+--------------+----------+----------+
|        1 | Laptop       |        1 | 55000.00 |
|        2 | Printer      |        1 | 12000.00 |
|        3 | Mouse        |        3 |  1500.00 |
|        4 | Monitor      |        1 | 15000.00 |
|        5 | Laptop       |        1 | 55000.00 |
+----------+--------------+----------+----------+
5 rows in set (0.04 sec)
```

**3. Create a View**

```
CREATE VIEW sales_report_view AS
SELECT c.customer_name, p.product_name, oi.quantity, oi.price
FROM order_item oi
JOIN sales_order o ON oi.order_id = o.order_id
JOIN customer c ON o.customer_id = c.customer_id
JOIN product p ON oi.product_id = p.product_id;
```

**Result Outcome:**

- The view sales_report_view was created successfully without any errors.
- Data was retrieved by combining information from the Customer, Product, Sales Order, and Order Item tables.
- The view displayed the following details:
  - Customer Name
  - Product Name
  - Quantity
  - Price
- JOIN operations correctly linked all tables using Customer ID, Order ID, and Product ID.
- The data shown in the view accurately reflected the records stored in the database.
- This confirms that the view creation and table relationships are working correctly.

```
mysql> SELECT * FROM sales_report_view;
+---------------+--------------+----------+----------+
| customer_name | product_name | quantity | price    |
+---------------+--------------+----------+----------+
| Amit Shah     | Laptop       |        1 | 55000.00 |
| Neha Verma    | Printer      |        1 | 12000.00 |
| Rahul Mehta   | Mouse        |        3 |  1500.00 |
| Pooja Singh   | Monitor      |        1 | 15000.00 |
| Karan Patel   | Laptop       |        1 | 55000.00 |
+---------------+--------------+----------+----------+
```

### 10.2 Achievements of the System

- Successfully created a structured relational database.
- Implemented CRUD operations (Create, Read, Update, Delete).
- Maintained data integrity using constraints.
- Implemented transaction handling using COMMIT and ROLLBACK.
- Generated reports using JOIN, GROUP BY, and VIEW.

---

### 10.3 Limitations

- The system does not include a graphical user interface (GUI).
- No real-time online payment integration.
- Basic security implementation.
- Limited to small-scale business operations.

---

### 10.4 Future Scope

- Develop a web-based or desktop application interface.
- Integrate online payment gateway.
- Implement advanced security and user authentication.
- Add automated billing and invoice generation system.
- Implement real-time stock alerts and notifications.
- Add dashboard analytics for better business insights.

---

# CHAPTER 11: SECURITY, BACKUP AND RECOVERY

## 11.1 Books Referenced


- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*.
- Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*.
- Date, C. J. *An Introduction to Database Systems*.


## 11.2 Online References


- MySQL Official Documentation – https://dev.mysql.com/doc/
- W3Schools SQL Tutorial – https://www.w3schools.com/sql/
- GeeksforGeeks Database Management System – https://www.geeksforgeeks.org/dbms/


## 11.3 Tools Used

- MySQL Database Server
- MySQL Workbench
- PlantUML (for UML diagrams)
- Microsoft Word (for documentation)

## 11.4 Research Papers / Articles (If Any)

- Various online articles related to Sales Order Processing Systems and Database Management concepts were referred for understanding system design and implementation.

# CHAPTER 12: FUTURE SCOPE AND CONCLUSION

## 12.1 Conclusion

The Sales Order Processing System has been successfully designed and implemented using structured database design principles and SQL queries. The system provides an organized and efficient method for managing customers, products, sales orders, and order items within a centralized database environment.

Throughout the development of this project, important database concepts such as normalization, primary keys, foreign keys, constraints, joins, subqueries, views, and transaction management were applied effectively. These concepts ensure data accuracy, integrity, and consistency across all tables. The system reduces manual errors, improves record maintenance, and enhances overall operational efficiency.

The implementation allows users to perform essential operations such as adding customers and products, placing orders, updating stock automatically, and generating sales reports. Advanced SQL queries such as GROUP BY, HAVING, and JOIN help in analyzing sales data and producing meaningful business insights.

This project also demonstrates the practical application of Database Management Systems (DBMS) in real-world business scenarios. It provides a strong foundation for understanding how relational databases support transaction processing systems.

In conclusion, the Sales Order Processing System meets all the specified objectives and requirements. It is reliable, structured, and scalable for small to medium-sized business environments. The project successfully integrates theoretical database concepts with practical implementation, making it a complete and functional database solution.

.

## 12.2 Future Scope

Although the system is functional, several improvements can be made in the future:

- Development of a user-friendly web or desktop interface.
- Integration of online payment gateway systems.
- Implementation of advanced security and authentication features.
- Addition of automatic invoice generation and email notifications.
- Real-time stock alerts and low inventory notifications.
- Dashboard analytics and graphical sales reports.
- Multi-user role management (Admin, Sales Staff, Manager).
- Cloud-based deployment for remote access.
- Integration with accounting software for automated financial tracking.
- Development of a mobile application for order management and real-time monitoring.
- Implementation of barcode or QR code scanning for faster product management.

# CHAPTER 13: REFERENCES

## 13.1 Books


- Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Pearson Education.
- Date, C. J. *An Introduction to Database Systems*. Pearson.


## 13.2 Online Resources


1. MySQL Official Documentation – https://dev.mysql.com/doc/
2. W3Schools SQL Tutorial – https://www.w3schools.com/sql/
3. GeeksforGeeks – Database Management System (DBMS) Articles – https://www.geeksforgeeks.org/dbms/
4. Oracle SQL Documentation – https://docs.oracle.com


## 13.3 Tools and Software Used

- MySQL Server
- MySQL Workbench
- PlantUML (for UML diagrams)
- Microsoft Word (for documentation preparation)

# CHAPTER 14: GLOSSARY

---

## 14.1 Database Terms

**Database** – A structured collection of related data stored electronically.

**DBMS (Database Management System)** – Software used to create, manage, and manipulate databases.

**SQL (Structured Query Language)** – A language used to manage and operate relational databases.

**Table** – A database object that stores data in rows and columns.

**Record (Row)** – A single entry in a table.

**Field (Column)** – A specific attribute or data item in a table.

---

## 14.2 Key Concepts

**Primary Key (PK)** – A unique identifier for each record in a table.

**Foreign Key (FK)** – A field that links one table to another table.

**Constraint** – A rule applied to a column to maintain data integrity.

**Normalization** – The process of organizing data to reduce redundancy.

**Relationship** – A logical connection between two or more tables.

---

## 14.3 SQL Operations

**SELECT** – Used to retrieve data from a table.

**INSERT** – Used to add new records to a table.

**UPDATE** – Used to modify existing records.

**DELETE** – Used to remove records from a table.

**JOIN** – Used to combine data from multiple tables.

**GROUP BY** – Used to group rows that have the same values.

**HAVING** – Used to filter grouped records.

**Subquery** – A query nested inside another query.

**View** – A virtual table based on a SQL query.

---

### 14.4 System Related Terms

**Sales Order** – A record representing a customer's purchase request.

**Order Item** – Individual product details within a sales order.

**Stock** – The quantity of products available.

**Transaction** – A sequence of database operations executed as a single unit.

**COMMIT** – Saves changes permanently in the database.

**ROLLBACK** – Cancels changes made during a transaction.

---